



ISIS ingestion, start to finish

How a raw detector dump becomes an image
we can measure shadows in, and what every line
the pipeline prints actually means

Gergő Csaba Morvai
Máni Mission terrain-hazard pipeline
September 2026

1 What this document is

This is the briefing for anyone who has to understand, run, or debug the HATI ingestion pipeline without a background in planetary data processing. It covers why the pipeline exists, what each piece of software in it does, what every line it prints on the console means, and what the code is doing while it prints them. No prior knowledge is assumed, and every technical term is defined the first time it appears and again in the glossary at the back.

The short version: NASA archives raw images from a camera in orbit around the Moon. Those files are not usable as pictures. This pipeline turns a chosen set of them into calibrated, map-projected images of the same patch of ground, all aligned with each other to better than one pixel, which is the input the hazard measurement needs.

Target site	84.7906° south, 29.1957° east
Instrument	Narrow Angle Camera, Lunar Reconnaissance Orbiter
Map scale	0.9 m per pixel, south polar stereographic
Frames usable	89 of the 696 the archive offers
Pass condition	median residual ≤ 1.00 px

2 Why the pipeline exists

HATI is a hazard map for lunar landing sites. It has to find rocks that are smaller than a pixel, at a place where the Sun never climbs more than a few degrees above the horizon.

The target here is a point near Nobile crater in the lunar south polar region, at 84.79° south. A lander touching down there needs to know whether the patch underneath it holds a 30 cm boulder. The best imagery available comes from the Narrow Angle Camera on Lunar Reconnaissance Orbiter, which resolves about half a metre to a metre per pixel from orbit. A 30 cm rock is therefore invisible. It occupies less than one pixel and changes that pixel's brightness by almost nothing.

So we do not look for the rock. We look for its shadow, which at this latitude is enormous.

2.1 The shadow trick

A vertical object of height h lit by a Sun at elevation angle e casts a shadow of length L :

$$L = h \cdot \cot(e) \quad \text{so} \quad h = L \cdot \tan(e) \quad (1)$$

Near the equator this is useless. The Sun climbs high, $\cot(e)$ falls toward zero, and a small rock casts almost no shadow. At 84.8° south the Sun tops out a few degrees above the horizon and the geometry inverts in our favour. At a Sun elevation of 5°, $\cot(5^\circ)$ is 11.4.

Object height (m)	Sun elevation (°)	Shadow length (m)	Shadow (px)	Measurable?
0.30	45	0.30	0.3	No
0.30	10	1.70	1.9	Marginal
0.30	5	3.43	3.8	Yes
0.30	2	8.59	9.5	Yes
1.00	5	11.43	12.7	Yes

A rock a third of a metre across, invisible in its own right, throws a dark streak four pixels long. That streak is measurable. The pipeline targets a Sun elevation band of 1.5° to 7.5° for exactly this reason, aiming at 5°, which is the compromise between long shadows and enough light to see anything at all.

In plain words

You cannot photograph the rock, so you photograph the stripe of darkness it throws across the ground. Near the pole the Sun is so low that the stripe is more than ten times longer than the rock is tall. The geometry does the magnification for you.

3 Why one image is not enough

Dark pixels are ambiguous. A shadow is dark. So is a patch of low-albedo soil, or a small depression. From a single frame there is no way to tell a hazard from a stain, and a hazard map built on stains would route a lander around obstacles that are not there and into ones that are.

The discriminator is motion. Move the Sun and a real shadow swings around its caster like a clock hand, staying anchored at the object's base. A dark patch of soil does not move at all, because it is painted on the ground.

That is the *solar sweep*. Collect frames of the same ground across a wide spread of Sun azimuths, find every dark blob in every frame, and cast a vote at the up-Sun end of each one, which is where the caster must be. Votes from a real obstacle pile up on a single point. Votes from an albedo patch smear into a ring, because the assumed caster position rotates with the Sun while the patch stays put.

Two consequences fall straight out of this, and the code enforces both. What matters is the spread of Sun azimuth rather than the number of frames, so the pipeline requires at least four frames covering at least 40° of azimuth and refuses to continue otherwise. And the frames must be aligned to sub-pixel accuracy, because the whole method rests on deciding whether a blob moved between two images.

Why alignment is the whole game

If two frames are misaligned by more than a pixel, then a shadow that appears to move by a pixel between them might be the shadow moving or might be the frame moving. The method cannot separate the two, and the measurement is worthless. That is why this pipeline ends in a hard pass-or-fail test rather than a picture.

4 What ISIS is, and why raw data is not a picture

ISIS is the Integrated Software for Imagers and Spectrometers, written and maintained by the USGS Astrogeology Science Center. It is the standard toolchain for turning planetary spacecraft data into something scientifically usable, and almost every published measurement from this camera passed through it.

What you download from the NASA Planetary Data System is an *EDR*, an Experiment Data Record. It is close to a raw dump of what the detector read out, and it has three properties that make it useless as an image.

- The numbers are detector counts, not brightness. They carry the sensor's dark current, its per-pixel gain variation, and an electronic echo from the readout. Two frames of identical ground taken on different orbits will not have the same values.
- There is no geometry attached. The file does not know where the spacecraft was, where it was pointing, or where the Sun was. It is a rectangle of numbers with a timestamp.
- The camera is a *line-scan* instrument. It has no shutter and takes no snapshot. It builds an image one row at a time as the spacecraft flies, so every row was taken from a slightly different place at a slightly different attitude. The raw frame is not a projection of anything.

ISIS fixes all three, in that order. It runs on Linux and macOS only, which is why this stage of HATI runs under WSL2 on a Windows machine.

One term worth having up front. A *cube* is ISIS's own file format, ending in `.cube`. It holds the image data together with a long text label carrying everything ISIS knows about the frame. *Level 1* means calibrated but still in camera geometry. *Level 2* means projected onto a map. The pipeline keeps only level 2 and deletes the intermediates, because each frame is a quarter of a gigabyte.

5 The ten stages

Every stage prints a line beginning `##STAGE` so the mission dashboard can follow progress without parsing English. Four of the ten are ISIS programs. The rest is HATI's own code.

	Stage	Owner	In one line
1	SELECT	HATI	Choose which frames to fetch
2	DOWNLOAD	HATI	Fetch the raw file from the archive
3	LRONAC2ISIS	ISIS	Translate the archive file into an ISIS cube
4	SPICEINIT	ISIS	Attach the spacecraft and Sun geometry
5	CAMPT	HATI	Ask the camera whether the site is on the detector
6	LRONACCAL	ISIS	Convert detector counts into real brightness
7	CAM2MAP	ISIS	Project the frame onto the map grid
8	COREGISTER	HATI	Measure how far the frame is from truth
9	GATE	HATI	Pass or fail the whole stack
10	MANIFEST	HATI	Hand the survivors to the shadow detector

5.1 Stage 1, SELECT

Reads the sweep catalogue and chooses which frames to fetch. It splits the compass into bins and picks one frame per bin, so the set covers many Sun directions rather than clustering at one. It rejects any frame whose footprint does not put the site comfortably inside the imaged strip, and keeps ranked alternates per bin in case the chosen frame turns out not to image the site after all. It refuses to continue on fewer than four frames or less than 40° of azimuth spread.

5.2 Stage 2, DOWNLOAD

Fetches the raw file from the LROC node of the Planetary Data System. Each frame is roughly 130 MB to 270 MB. Files already on disk are reused, so a re-run costs no bandwidth.

5.3 Stage 3, LRONAC2ISIS

Unpacks the archive file into an ISIS cube. This is a format translation and nothing more. It reads the archive label, decompresses the detector data, and writes a `.cub` with an ISIS label. The pixel values are still raw counts and there is still no geometry.

```
lronac2isis from=M1496181900LE.IMG to=M1496181900LE.cub
```

5.4 Stage 4, SPICEINIT

Attaches the geometry. SPICE is NASA's system for describing where everything was and how it was oriented at a given instant, distributed as binary files called *kernels*. This step works out, for every image line, where the spacecraft was, which way the camera pointed, where the surface sat underneath it, and where the Sun was. It writes all of that into the cube's label. Nothing downstream can work without it.

The full kernel set runs well past 100 GB, so `web=yes` queries the USGS kernel service instead of keeping a local copy. This is the slowest step per frame and it needs a working internet connection.

```
spiceinit from=M1496181900LE.cub web=yes
```

5.5 Stage 5, CAMPT

The first moment we can ask the camera itself a question. `campt` in ground mode takes a latitude and longitude and reports which detector sample and line it falls on. If the site is not on this frame's detector we learn it here, before spending the expensive steps on a frame that cannot help.

The answer needs bounding. A camera channel is 5064 samples wide, and `campt` will report sample 6320 without complaining, because it extrapolates past the edge of the focal plane. So the

test is not whether the sample is positive. It is whether the sample sits inside the real detector with the correlation window's worth of room to spare.

```
campt from=M1496181900LE.cub type=ground latitude=-84.7906 longitude=29.1957
```

5.6 Stage 6, LRONACCAL

Radiometric calibration. It subtracts the dark level, divides out the flat field so every detector element responds the same way, removes the readout echo, and converts counts into a physical reflectance quantity. After this a brightness value means the same thing in a 2013 frame as in a 2025 one, which is what makes frames from different years comparable at all.

```
lronaccal from=M1496181900LE.cub to=M1496181900LE.cal.cub
```

5.7 Stage 7, CAM2MAP

The step that makes pixels comparable in space. It takes the camera geometry from stage 4 and resamples the frame onto a fixed map grid, so a given pixel in the output always corresponds to the same patch of ground no matter which orbit the frame came from. HATI's grid is south polar stereographic at 0.9 m per pixel, defined in a small text file the script writes itself.

Polar stereographic is used because ordinary latitude and longitude are unusable this close to the pole, where the meridians converge and a few hundred metres of ground can span tens of degrees of longitude. The output is a level 2 cube, and it is what everything downstream reads.

```
cam2map from=...cal.cub map=sweep_polar.map to=...lev2.cub pixres=map  
defaultrange=map
```

5.8 Stage 8, COREGISTER

Measures how far each projected frame sits from truth. It cuts an 800 by 800 pixel window centred on the touchdown out of both the new frame and a trusted reference image, then cross-correlates the two to find the offset between them to a tenth of a pixel. Then it applies that offset and correlates again to see what is left over. It writes `coreg_report.csv`.

5.9 Stage 9, GATE

The verdict. It takes the median leftover error across all measured frames and passes at or below one pixel. This is the single number that decides whether the shadow measurement is allowed to run at all.

5.10 Stage 10, MANIFEST

Writes `manifest.json`, listing every surviving frame with its Sun angle, its file path, and its measured shift. That file is the handover to the shadow detector.

6 Reading the console

Every progress line has the same shape, which makes the whole log skimmable once you know it.

```
##STAGE  NAME  STATE  detail
```

The state is one of four words. `run` means it started. `ok` means it finished. `fail` means it did not, and the detail says why. `skip` means it was deliberately not attempted, which during a dry run is every stage.

6.1 The opening lines

Selection

```
sweep CSV: solar_sweep_-84.791_+29.196_EDRNAC4.csv
(89 usable frames; 541 miss the site, 66 clip it by under 600 m)
##STAGE SELECT run
pid          az(proxy)  elev    margin  alts
nac.m1496069741le  105.7   4.00    3149 m    3
nac.m1496181900le  121.5   4.77    2371 m    3
nac.m1496434319le  157.2   5.36    1327 m    3
nac.m1496630688le  184.9   4.41    2345 m    3
nac.m1433078591re  217.9   3.57    1665 m    3
##STAGE SELECT ok 5/10 bins, azimuth spread 112 deg
```

The archive offered 696 frames whose bounding box touches the site. Only 89 of those actually image it with room to spare. The five picked cover 112° of Sun azimuth, which clears the 40° minimum comfortably. **margin** is how far inside the imaged strip the site sits, in metres. **alts** is how many backup frames that bin is holding in reserve.

6.2 A frame going through cleanly

Per-frame chain

```
##STAGE DOWNLOAD run M1433078591RE.IMG
... 67 MB
##STAGE DOWNLOAD ok M1433078591RE.IMG (132 MB)
##STAGE LRONAC2ISIS ok M1433078591RE
##STAGE SPICEINIT ok M1433078591RE
##STAGE CAMPT ok M1433078591RE: site at sample 1548 of 5064, line 33725
##STAGE LRONACCAL ok M1433078591RE
##STAGE CAM2MAP ok M1433078591RE
```

The CAMPT line is the one to actually read. Sample 1548 of 5064 means the site lands well inside the detector's width, which is what we want. Line 33725 is how far down the strip it sits, and these strips are tens of thousands of lines long, so a large number there is normal. This was the one frame of the 4 September run that went all the way through.

6.3 A frame rejected before it costs anything

Rejection and fallback

```
##STAGE SPICEINIT ok M1496434319LE
##STAGE CAMPT fail M1496434319LE: site at sample 6053 of 5064
-- 989 past the edge of this channel
-> M1496434319LE does not put the site on its detector;
    trying M1496462370LE
##STAGE DOWNLOAD run M1496462370LE.IMG
```

This is the gate working as intended. The footprint said this frame covered the site by 1327 m, but the camera model says the site falls 989 samples beyond the edge of the detector. The footprint describes the whole observation and cannot see the boundary between the camera's two optics. The run drops the frame and reaches for that bin's next alternate instead of losing the bin.

6.4 A co-registration that measures

Illustrative, not a captured run

The gate has not been passed yet at the time of writing. The block below shows what a pass looks like. The shape of the output and the meaning of every field are exact, but the numbers in it are not measured results.

Coregister and gate

```
##STAGE COREGISTER run
##STAGE COREGISTER ok nac.m1433078591re
      shift=(-24.10,-58.30) px  residual=0.41 px  ncc=+0.612
co-registration budget -> data/sweep/coreg_report.csv

frames measured      : 5 of 5
median raw shift     : 63.10 px  (57 m at the 0.9 m/px map scale)
  -- this is the SPICE pointing error, which co-registration
     exists to remove. It is not the gate.
median residual      : 0.41 px  <-- THE GATE, passes at <= 1.00
median aligned ncc    : +0.612
##STAGE GATE ok median residual 0.41 px over 5 frames
```

Three numbers, and they mean different things. **shift** is where the frame was found relative to the reference, and a large value here is expected and fine. **residual** is what remains after correcting it, and that is the gate. **ncc** is how similar the two aligned images are, on a scale from -1 to $+1$. A middling ncc with a small residual is the normal result for a solar sweep, because the two frames were lit from different directions and genuinely contain different shadows.

6.5 A co-registration that fails, and says why

Diagnosed failure

```
##STAGE COREGISTER fail nac.m14966306881e: only 0% real data in the
      window; this frame's strip may not cover the site
the whole cube is empty (0.0% valid) -- cam2map wrote no image data,
so this is a projection/extent problem, not selection
```

The second line is the important one. An empty window has three possible causes that need opposite fixes, so the pipeline works out which one it is rather than leaving you to guess. It reports the cube's fill percentage, the distance from the touchdown to the nearest real pixel, and one of three verdicts: the projection produced nothing, the frame genuinely misses the site, or the window is being read in the wrong place.

6.6 Every failure line in one place

Line	What it means	What to do
SELECT fail	Fewer than four frames, or under 40° of azimuth spread	Widen the elevation band, or lower --n
DOWNLOAD fail	The archive did not serve the file	Usually transient. Re-run; finished downloads are cached
SPICEINIT fail	The kernel service was unreachable, or the frame has no kernels	Check the connection and re-run
CAMPT fail	The site is not on this frame's detector	Nothing. The run moves to the next alternate by itself
CAMPT skip	campt is missing or gave an unreadable answer	Nothing, but the footprint is now the only check, which is weaker
CAM2MAP fail	Projection failed, often an extent that excludes the frame	Check the latitude and longitude range in the map file
COREGISTER fail	The window could not carry a measurement	Read the diagnosis line printed underneath it
GATE fail	Median residual is above one pixel	Stop. Do not run the shadow measurement on this stack

7 The gate, and why it is the residual

Co-registration produces two error numbers and they are easy to confuse. Getting this backwards was a real mistake in this project, so it is worth stating plainly.

Number	What it is	Typical size	Gated
shift	Where the frame was found relative to the reference. The residual error in the spacecraft's own knowledge of its position and pointing	tens of px	no
residual	What is left after applying the shift. Our own alignment error	under 1 px	yes
ncc	How similar the two aligned images look, from -1 to +1	0.3 to 0.8	no

The shift is the thing co-registration exists to remove. SPICE geolocation for this camera is good to some tens of metres before any bundle adjustment, so a shift of 60 pixels at 0.9 m per pixel is an ordinary, healthy result. Gating on it would throw away perfectly correctable frames.

What has to be small is what survives the correction. If two frames are still misaligned by more than a pixel after correction, then a shadow appearing to move by a pixel between them could just as easily be the frame that moved.

The rule

Median residual at or below 1.00 px passes. Above it, stop and report it. Do not run the shadow measurement on a stack that failed the gate, and do not adjust the threshold to make a run pass.

The ncc is diagnostic rather than a pass condition, and it needs reading with care. A low ncc alongside a small residual is not a problem. It is the expected signature of a solar sweep, where two frames of the same ground were lit from 100° apart and therefore genuinely contain different shadows. A low ncc alongside a large residual is a different matter, and means the correlation did not find the right peak.

8 What the code does

Two scripts, about 900 lines together. The first asks the archive what exists. The second turns a subset of it into aligned images.

8.1 `solar_sweep_query.py`

Asks the Orbital Data Explorer, NASA's query service for planetary archives, for every frame whose footprint touches the site, then tabulates the illumination geometry of each. The distribution of Sun azimuths across the results is the solar sweep that is actually available for that place, which is worth knowing before committing to any downloads.

The function that matters is `site_margin_m`, which answers how far inside a frame's imaged strip the site sits. It projects the footprint polygon's corners into metres, then measures the distance from the site to the nearest polygon edge, positive inside and negative outside. Projecting first is the entire point. Testing the polygon in raw degrees looks reasonable and is wrong near the pole, where a footprint edge spanning tens of degrees of longitude is a straight line to the test and a kilometres-wide arc on the ground. The script writes one row per frame including a `margin_m` column, and keeps only frames at least 600 m clear by default.

8.2 `ingest_sweep.py`

The pipeline itself, in reading order.

Function	What it does
<code>load_csv</code>	Reads the sweep catalogue and drops frames that miss the site or only clip it. Keeps a separate unfiltered index of everything, because a frame rejected on its own footprint can still be the sibling channel that rescues its partner.
<code>select_frames</code>	Bins by Sun azimuth and picks the best frame per bin, ranking by nearness to the target elevation with a tie-break toward frames further inside the strip. Attaches ranked alternates. Enforces the four frame and 40° minimums.
<code>download</code>	Streams the raw file from the archive, skipping anything already on disk above 10 MB.
<code>isis</code>	One-line wrapper that runs an ISIS program as a subprocess and returns success plus the last 400 characters of output, which is what the failure detail in the log comes from.
<code>cube_dims</code>	Reads the real sample and line counts out of the ISIS label, so the detector bound is measured rather than assumed.
<code>campt_covers</code>	The authoritative coverage gate. Runs <code>campt</code> in ground mode, parses the sample and line, and returns one of four verdicts: inside, off the sample edge, off the readout, or unknown. Unknown defers to the footprint rather than discarding a possibly good frame.
<code>sibling_channel</code>	Swaps the camera's left optic for its right and back. The two optics image adjacent ground swaths, so a site just past the edge of one is often well inside the other.
<code>lev2_has_site</code>	Checks whether an already-projected cube actually holds pixels at the touchdown. Run on every cached product, so a known-empty file rebuilds itself instead of being trusted forever.
<code>process_frame</code>	Drives the ISIS chain for one frame, with the <code>campt</code> gate placed between the two cheap steps and the two expensive ones. Deletes the intermediates on success.
<code>where_is_the_data</code>	Runs only when a window comes back empty. Reads the cube decimated, finds every valid pixel, and reports which of the three causes it is.
<code>coregister</code>	Cuts matching windows, masks missing pixels, mean-subtracts both, cross-correlates for the shift, applies it, correlates again for the residual, and computes the correlation of the aligned pair.
<code>main</code>	Parses options, runs selection, and either writes a dry-run plan or executes. Walks each bin's queue until one frame survives.

8.3 Two details inside `coregister` worth knowing

ISIS marks missing pixels with negative `FLT_MAX`, which is about -3.4×10^{38} . That value is a perfectly finite number as far as a floating point check is concerned, so it passes an `isfinite` test and then overflows the variance to infinity, and the correlation returns an exact zero shift with a not-a-number error. That looks exactly like a perfect alignment. It is now masked explicitly, and any window with less than half real data is refused outright.

The correlation runs unnormalised rather than with the library's default phase normalisation. Phase normalisation whitens the frequency spectrum, which amplifies the highest-frequency content. On imagery that has been resampled, and `cam2map` resamples by interpolating, that content is numerical noise rather than signal, and the correlation peak disappears. A deliberately planted three pixel shift comes back as exactly zero. Unnormalised correlation recovers planted shifts to a tenth of a pixel on smooth, noisy, sharply textured and contrast-flipped test images alike.

8.4 Tests

Three test files, all offline, all runnable in seconds. `test_footprint.py` plants strips of known geometry and pins the containment arithmetic against the two real archive footprints that first exposed the bug. `test_coreg.py` plants known shifts and demands them back to a fifth of a pixel, including the case the library default silently returns zero for. `test_channels.py` covers the terrain channel stack, which is a different part of HATI.

9 Four ways this has already failed

Worth knowing, because most of the guards in the code exist to catch one of these, and their messages look cryptic without the history.

9.1 Correlating against nothing, and calling it perfect

The first real run reported a shift of exactly (0.00, 0.00) with a not-a-number error on every frame. That reads as flawless alignment. It was the signature of correlating a window that contained no image data, because the missing-pixel sentinel had not been masked. Anything that cannot carry a real measurement is now refused rather than reported as a pass.

9.2 Trusting a bounding box

The archive's spatial query filters on the footprint's bounding box. These strips are long, narrow and diagonal, so near the pole the bounding box is enormously larger than the strip. Three of the first four frames imaged nothing at the site.

9.3 Testing the polygon in the wrong coordinate space

The fix for that was to test the footprint polygon instead of its box, which was right in principle and wrong in execution: the test ray-cast in raw degrees. At 84.8° south that space is stretched about tenfold in longitude, the polygon balloons, and nearly everything passes. All five frames of the next run passed the test and all five came back empty. Measured properly they missed the site by 0.4 km to 3.0 km. The test now projects the vertices first and returns a distance in metres.

9.4 Asking the camera the right question without bounding the answer

Then came the `campt` gate, which checked only that the sample number was positive. A camera channel is 5064 samples wide and `campt` extrapolates past the focal plane without erroring. The run put the site at samples 5069, 6053 and 6320 on three frames whose footprints read as 1.3 km to 2.3 km inside, and all three projected to empty. The sample is now tested against the cube's real width.

The pattern

Every one of these produced output that looked like success. That is why the pipeline now prefers a loud refusal over a plausible number, and why the gate is printed as a verdict rather than left in a spreadsheet for someone to interpret.

10 Running it

Selection and the dry run work anywhere Python does. Everything from stage 3 onward needs ISIS, so on Windows that means inside WSL2.

10.1 Find out what imagery exists

```
python scripts/solar_sweep_query.py --lat -84.7906 --lon 29.1957 --halfwidth-km 3
```

Writes the sweep catalogue and prints how many frames actually image the site. Costs one web request and no disk.

10.2 See the plan without downloading anything

```
python scripts/ingest_sweep.py --n 10
```

The dry run is the default. It prints the frames it would fetch and writes `data/sweep/plan.json`.

10.3 Actually run it

```
python scripts/ingest_sweep.py --n 10 --execute
```

Budget roughly 0.35 GB and several minutes per frame, most of it in `spiceinit` and `cam2map`.

10.4 Options worth knowing

Option	Default	Effect
<code>--n</code>	10	Azimuth bins, and so the maximum number of frames
<code>--min-elev</code>	1.5	Lower edge of the Sun elevation band, in degrees
<code>--max-elev</code>	7.5	Upper edge. Widen the band if selection cannot find enough frames
<code>--target-elev</code>	5.0	Preferred Sun elevation within a bin
<code>--min-margin-m</code>	600	How far inside the strip the site must sit
<code>--alternates</code>	3	Backup frames per bin for <code>campt</code> to fall back on
<code>--frames</code>	none	Force specific product ids and bypass the gates, for debugging
<code>--no-campt</code>	off	Skip the ground-point check. Only for isolating a problem
<code>--execute</code>	off	Without it, nothing downloads and no ISIS program runs

10.5 What to read afterwards

The `##STAGE GATE` line is the answer. `data/sweep/coreg_report.csv` holds the per-frame numbers behind it, and `data/sweep/manifest.json` is what the shadow measurement consumes next.

11 Glossary

EDR

Experiment Data Record. The rawest archived form of a frame, close to a detector dump.

DN

Digital Number. A raw detector count, before calibration. Not a brightness.

cube

ISIS's file format. Image data plus a long text label holding everything ISIS knows about the frame.

level 1 and level 2

Calibrated but still in camera geometry, versus projected onto a map grid.

SPICE

NASA's system for spacecraft and planetary geometry: positions, orientations and time.

kernel

One SPICE data file. The complete set for this spacecraft runs past 100 GB, so we query a web service instead.

PDS

Planetary Data System. NASA's archive, where the raw files are downloaded from.

ODE

Orbital Data Explorer. The search interface over the archive that the sweep query talks to.

NAC

Narrow Angle Camera on Lunar Reconnaissance Orbiter. Two optics, each 5064 pixels wide, imaging adjacent swaths.

line-scan camera

A camera with no shutter that builds an image one row at a time as the spacecraft moves.

footprint

The archive's polygon describing roughly what ground an observation covers. Coarse, and it describes the observation rather than one optic.

ortho

An image resampled so every pixel sits at its true map position, with terrain distortion removed.

polar stereographic

A map projection that behaves near a pole, where latitude and longitude do not.

phase correlation

A method for finding the offset between two images by comparing them in the frequency domain.

nodata

A reserved pixel value meaning nothing was imaged here.

PVL

Parameter Value Language. The plain-text label format ISIS uses, including the map definition file.

solar sweep

A set of images of one place taken at many different Sun angles. The core of the method.

Pipeline source: `scripts/solar_sweep_query.py` and `scripts/ingest_sweep.py`. Tests: `tests/test_footprint.py`, `tests/test_coreg.py`.